

Presentation Delivery Script

Experimental Analysis of Sorting Algorithms

Maria Maiorescu
West University of Timișoara
May 18, 2026

Slide 1 — Title and Introduction

Time: 30 seconds

Support Material:

- Presentation title
- Author name
- University

Method of Presentation:

Short oral introduction.

What I Say:

Today I will present my research paper called *Experimental Analysis of Sorting Algorithms*.

The goal of this project was to compare six classical sorting algorithms using different array sizes and different types of input data.

The paper combines theoretical analysis with practical benchmarking implemented in C. Also, the implementation is available on GitHub (add a QR code).

Organizational Remarks:

Introduce the topic and move to the next slide.

Slide 2 — Importance of Sorting

Time: 30 seconds

Support Material:

- Examples:
 - databases
 - search engines
 - operating systems

Method of Presentation:

Explanation with simple examples.

What I Say:

Sorting is one of the most important problems in computer science.

It appears in databases, search engines, operating systems, and many other applications.

Although textbooks focus mainly on theoretical complexity, real performance also depends on factors such as input structure and implementation details.

Because of this, practical benchmarking is important in addition to theoretical analysis.

Organizational Remarks:

Maintain eye contact and avoid reading directly from the slide.

Slide 3 — Related Work

Time: 20 seconds

Support Material:

- Knuth
- Cormen, Leiserson, Rivest, Stein
- TimSort

Method of Presentation:

Short overview of related studies and images

What I Say:

This project was inspired by several important studies about sorting algorithms.

Knuth and the Cormen, Leiserson, Rivest, Stein textbooks provide the theoretical background, while more recent studies focus on adaptive sorting methods such as TimSort.

My project compares six algorithms using the same hardware and the same testing conditions.

Organizational Remarks:

Keep this section short.

Slide 4 — Algorithms Used

Time: 1.5 minutes

Support Material:

- Complexity table

- Names of algorithms

Method of Presentation:

Technical explanation.

What I Say:

In this project, I tested six classical sorting algorithms: Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort, and Radix Sort.

Bubble Sort, Insertion Sort, and Selection Sort become slow when the input size grows.

Merge Sort and Quick Sort perform much better on large arrays.

I also included Radix Sort, which was the fastest algorithm in most large tests.

Organizational Remarks:

Point briefly at the complexity table.

Slide 5 — Benchmark Methodology

Time: 2 minutes

Support Material:

- Benchmark structure
- Input variants
- Timing method

Method of Presentation:

Step-by-step explanation.

What I Say:

The benchmarking system was implemented entirely in C.

The program contains: data generation, sorting functions, timing functions, and benchmarking code.

I tested random arrays, reversed arrays, and partially sorted arrays.

The execution time was measured using the `clock()` function.

For small arrays, the timings were averaged over many runs to reduce noise.

Organizational Remarks:

Slow down slightly because this slide contains technical details.

Slide 6 — Experimental Setup

Time: 45 seconds

Support Material:

- Array sizes
- GCC compiler
- Optimization level

Method of Presentation:

Short technical overview.

What I Say:

The experiments were executed on a personal laptop using GCC with optimization level O2.

The tested array sizes ranged from 10 elements up to one million elements.

Each algorithm was tested on different input types to observe how scalability and input structure influence performance.

Organizational Remarks:

Transition toward the results section.

Slide 7 — Results for Small and Medium Arrays

Time: 1 minute

Support Material:

- Tables
- Small graphs

Method of Presentation:

Comparative analysis.

What I Say:

For small arrays, the differences between algorithms are smaller.

However, Radix Sort and Merge Sort already perform very well.

Insertion Sort also becomes surprisingly efficient when the input is almost sorted.

For medium arrays, the quadratic algorithms start becoming much slower, while Merge Sort, Quick Sort, and Radix Sort scale much better.

Organizational Remarks:

Focus on trends instead of exact numbers.

Slide 8 — Results for Large Arrays

Time: 1 minute 20 seconds

Support Material:

- Large graphs
- Logarithmic plots

Method of Presentation:

Graph interpretation.

What I Say:

For large arrays, the differences become very large.

Bubble Sort and Selection Sort become impractical because of their quadratic complexity.

Merge Sort remains stable and predictable, while Radix Sort becomes the fastest algorithm for large inputs.

Quick Sort performs very well on random data, but some structured inputs still produce slowdowns.

The logarithmic graphs clearly show the difference between quadratic growth and more efficient algorithms.

Organizational Remarks:

Briefly explain how to read the graph axes.

Slide 9 — Analysis and Problems Encountered

Time: 2 minutes

Support Material:

- Key findings
- Quick Sort behaviour

Method of Presentation:

Discussion of important observations.

What I Say:

One interesting result was that Insertion Sort became much faster when the input was almost sorted.

Another important observation was the behaviour of Quick Sort on structured inputs.

At first, Quick Sort caused a stack overflow because the pivot selection produced very deep recursion. After changing the implementation to a median-of-three pivot, the algorithm became much more stable.

This shows that practical implementations can behave differently from theoretical expectations.

Organizational Remarks:

Point at the results.

Slide 10 — Conclusion and Future Work

Time: 1 minute

Support Material:

- Summary bullets
- Future improvements

Method of Presentation:

Final summary.

What I Say:

In conclusion, this project compared six classical sorting algorithms using different array sizes and input types.

Bubble Sort and Selection Sort became very slow on large inputs, while Merge Sort and Radix Sort remained efficient.

The experiments also showed that input structure strongly influences algorithms such as Insertion Sort and Quick Sort.

For future work, I would like to test more data types and compare these algorithms with standard library sorting functions.

Thank you for your attention!

Organizational Remarks:

End the presentation clearly.